

Self-Contained MHTML Artifact Architecture with A2UI

Production Framework: Compiling Dynamic Layouts, Inline SVG Serializations, and A2UI Core Integration

Document Type: Technical Architecture Specification	Target Integration: CopilotKit v2, AG-UI Transport	Design Standards: A2UI (Agent-to-User Interface) Specification
--	---	---

1. Executive Summary & Architectural Pivot

This architecture details a robust approach for generating on-demand, downloadable data portfolios using **MHTML (MIME HTML)** instead of standard external PDF binaries. This configuration enables enterprise applications to satisfy dual-trigger requirements—where the artifact is initialized via a physical UI button or requested programmatically inside a conversation loop.

Rather than burning heavy backend compute to compile canvas frames or risk credential disclosure on client devices, the **CopilotKit v2 / AG-UI** transport layer works in tandem with **A2UI (Agent-to-User Interface)**. The agent emits schema-validated interface intent instead of executable presentation scripts. The host system maps these payloads to its own trusted design system and bundles HTML, layout CSS, and vectorized analytics into a single, offline-viewable MHTML file that opens perfectly within standard corporate office software (e.g., Microsoft Word, browsers).

2. The Strategic Role of A2UI (Agent-to-User Interface)

A2UI establishes an absolute abstraction contract between the AI model's internal data generation capabilities and the host platform's layout configuration engine. It functions as the blueprint, defining exactly how the system behaves across different operation spaces.

- Structural Protocol Contract:** The agent does not emit raw code or design parameters. It leverages a structured JSON catalog that outlines layout intention (e.g., side-by-side matrices, multi-series data tables).
- Form-Driven Interface Streamlining:** A2UI resolves the "chat bottleneck" scenario. Instead of prompting users sequentially across 5 text threads to configure parameters for a report, the agent streams a declarative interface form. The user populates inputs concurrently, returning a clean data matrix payload to the execution path.
- Zero-Trust Injection Guard:** By isolating layout execution from the agent, malicious cross-site scripting (XSS) vectors are eliminated. The host application maintains an immutable component allowlist (e.g., `LineChart`, `MetricCard`). Unrecognized definitions from the agent are automatically intercepted and safely filtered out.
- Design System Preservation:** The host interface translates A2UI definitions using native Tailwind classes, glassmorphism layers, or animated waves, ensuring the offline artifact precisely reflects company design standards.

3. Production CopilotKit v2 Action Registration

The client-side infrastructure registers the action hook within the execution runtime. This intercept engine captures structural layouts and processes execution states gracefully.

```
import { useCopilotAction } from "@copilotkit/react-core";
import { generateAndDownloadMHTML } from "@utils/mhtmlBuilder";

export function MhtmlReportBridge() {
```

```

useCopilotAction({
  name: "generateAnalyticalReportArtifact",
  description: "Compiles structural sections containing text columns, tables, and
visualization charts into an offline MHTML file.",
  parameters: [
    {
      name: "reportTitle",
      type: "string",
      required: true,
      description: "The header title of the document"
    },
    {
      name: "sections",
      type: "object",
      required: true,
      description: "A2UI layout sections array detailing textual descriptions, charts, and
inline data."
    }
  ],
  render: ({ args, status }) => {
    return (
      <div className="p-4 border rounded-xl backdrop-blur-md bg-white/10 space-y-3">
        <div className="flex items-center justify-between">
          <h4 className="font-semibold text-sm">{args.reportTitle} || "Processing
Blueprint..."</h4>
          <span className="text-xs px-2 py-0.5 rounded-full bg-blue-500/20 text-blue-400
animate-pulse">
            {status === "executing" ? "Compiling Data Structure" : "Ready"}
          </span>
        </div>

        {status === "completed" && (
          <button
            onClick={() => generateAndDownloadMHTML(args.reportTitle, args.sections)}
            className="w-full py-2 px-3 bg-gradient-to-r from-blue-600 to-indigo-600 text-
white rounded-lg font-medium text-xs shadow-md transition-all"
          >
            Download Self-Contained MHTML Report
          </button>
        )}
      </div>
    );
  },
  handler: async (args) => {
    return { success: true, compiledTimestamp: new Date().toISOString() };
  },
});

return null;
}

```

4. Core MHTML Compilation Engine

To ensure visualizations remain persistent outside of live sandbox environments, the rendering module loops over vector SVGs, serializes them to base64 images, strings styles together, and builds a strict, multipart MIME file block.

```
export function generateAndDownloadMHTML(title: string, sections: any[]) {
  const workspaceElement = document.getElementById("capture-workspace-root");
  if (!workspaceElement) {
    console.error("Target preview DOM reference unavailable.");
    return;
  }

  // 1. Deep clone DOM tree to insulate active view during translation
  const clonedDoc = workspaceElement.cloneNode(true) as HTMLElement;

  // 2. SVG-to-Base64 Image Interceptor Engine
  const svgElements = clonedDoc.querySelectorAll("svg");
  svgElements.forEach((svg) => {
    const svgString = new XMLSerializer().serializeToString(svg);
    const svgBase64 = window.btoa(unescape(encodeURIComponent(svgString)));

    const img = document.createElement("img");
    img.src = `data:image/svg+xml;base64,${svgBase64}`;
    img.style.width = svg.getAttribute("width") || "100%";
    img.style.height = svg.getAttribute("height") || "auto";

    svg.parentNode?.replaceChild(img, svg);
  });

  // 3. Consolidate Active Application Stylesheets
  let compiledCss = "";
  document.querySelectorAll("style, link[rel='stylesheet']").forEach((sheet) => {
    try {
      if (sheet.tagName === "STYLE") {
        compiledCss += sheet.textContent;
      } else if (sheet instanceof HTMLLinkElement && sheet.sheet) {
        compiledCss += Array.from(sheet.sheet.cssRules).map(rule => rule.cssText).join("
");
      }
    } catch (e) { /* Suppress Cross-Origin style isolation triggers */ }
  });

  const htmlBody = clonedDoc.innerHTML;
  const mimeBoundary = "----_Part_Enterprise_Compilation_Boundary";

  // 4. Construct Multi-Part Specification Block
  const mhtmlPayload = [
    `From: `,
    `Subject: ${title}`,
    `Date: ${new Date().toUTCString()}`,
    `MIME-Version: 1.0`,
    `Content-Type: multipart/related; boundary="${mimeBoundary}"; type="text/html"`,
    ``,
    `--${mimeBoundary}`,
    `Content-Type: text/html; charset="utf-8"`,
  ]
```

```

`Content-Transfer-Encoding: quoted-printable`,
`
`,
`<head>`,
`<meta charset="utf-8">`,
`<title>${title}</title>`,
`<style>${compiledCss}</style>`,
`</head>`,
`${htmlBody}`,
`
`,
`
`,
`--${mimeBoundary}--`
].join("
");

// 5. Instantiate Direct Binary File Transfer Anchor
const fileBlob = new Blob([mhtmlPayload], { type: "message/rfc822" });
const downloadUrl = URL.createObjectURL(fileBlob);

const anchor = document.createElement("a");
anchor.href = downloadUrl;
anchor.download = `${title.toLowerCase().replace(/\s+/g, "_")}.mhtml`;
document.body.appendChild(anchor);
anchor.click();

document.body.removeChild(anchor);
URL.revokeObjectURL(downloadUrl);
}

```

5. Double-Trigger Framework: Button and Chat Integration

The dual-pane environment permits seamless operation orchestration across layout contexts. Users can request updates conversationally via chat or invoke structural evaluations by directly triggering button handlers.

Manual Execution Implementation Pattern

When utilizing physical triggers (such as control room macro buttons), developers must pass execution context arguments directly into CopilotKit's execution manager. This updates the A2UI layout state instantly without requiring intermediate message generation loops.

```

import { useCopilotContext } from "@copilotkit/react-core";

export function ManualExecutionDashboardTrigger() {
  const copilotContext = useCopilotContext();

  const handleManualCompilation = async () => {
    // Programmatic activation via A2UI schema parameters contract
    await
    copilotContext.currentActionExecutionWindow?.runAction("generateAnalyticalReportArtifact", {
      reportTitle: "On-Demand Infrastructure Footprint Assessment",
      sections: [
        {
          id: "man_sec_101",
          columns: [

```

```

        { width: "1/2", content: [{ type: "text", content: "Cluster auto-scaling profiles
updated. Overhead thresholds balanced gracefully." }] },
        { width: "1/2", content: [{ type: "chart", chartType: "line", data: [{ label: "Node
1", value: 45 }, { label: "Node 2", value: 52 }] }] }
    ]
  }
}
});
};

return (
  <button
    onClick={handleManualCompilation}
    className="px-4 py-2 bg-slate-900 text-white border border-slate-700 rounded-lg text-xs
font-semibold hover:bg-slate-800 transition-all shadow-sm"
  >
    ⚡ Force Synthesize Report Profile
  </button>
);
}

```

6. Production Strategy & Validation Lifecycle

Engineering Phase	Identified Validation Constraint	Architecture Remediation & Mitigation Strategy
SVG Serialization	Missing <code>xmlns</code> identifiers cause document generation drop-offs outside standard browsers.	Inject <code>xmlns="http://www.w3.org/2000/svg"</code> programmatically inside the clone mutation sequence before compilation begins.
Layout Isolation	CSS variables containing layout values fail to compile or read inside native text applications.	Utilize a post-processing filter that converts dynamic CSS custom properties into standard pixel or rem declarations inside the document head.
Component Validation	Malicious prompt engineering injecting cross-site execution nodes into flat blocks.	Run inbound A2UI JSON structures against runtime type validators (Zod schemas) before pushing definitions to child workspace modules.